

**AD9081 / AD9082 / AD9986 / AD9988 / AD9207 / AD9209 /
AD9177
API Specification,
Integration and Porting Guide.**

Rev 1.7.0

Table of Contents

Revision History	5
Introduction	7
Purpose	7
Scope.....	7
Disclaimer	7
adi_adxxxx.h	8
Data Structures	8
hal_info.....	8
dev_info.....	9
serdes_info.....	9
clk_info	9
API Overview Block Diagram	9
Struct Reference	11
ADxxxx API Functions Outline	12
ADxxxx API Functions Detailed Descriptions	26
1.0 Device Initialization & Clock Configuration	26
1.1 Device Init / De-init and Hardware Platform API.....	27
1.2 Block Level Clock API.....	30
2.0 Transmit Path Setup	32
2.1 Transmit TxEn.....	33
2.2 Transmit DAC Analog Core	35
2.3 Transmit Channelizer Gain	36
2.4 Transmit Datapath Setup	37
2.5 Transmit Path NCOs.....	38
2.5.1 Transmit Path FFH.....	41
2.6 Transmit Path PA Protection.....	43
2.7 Transmit Path Helper APIs	48
3.0 Receive Path Setup	51
3.1 Receive Datapath Setup	54
3.2 Receive Fast Detect.....	55
3.3 Receive Programmable Filter.....	58
3.3.1 Low Level Receive Programmable Filter API	62
3.4 Receive Path NCOs	67
3.4.1 Receive Path NCO Delay API	77
3.5 Receive Path Decimation and C2R.....	81
3.6 Receive Path Gain.....	83
3.7 Receive Path Helper APIs	83
4.0 SERDES Link Establishment & Monitoring	93
4.1 SERDES Receiver Link Setup	96
4.2 SERDES Transmitter Link Setup.....	103

Evaluation Warning: The document was created with Spire.Doc for Python.

Appendix: Additional Digital Feature Blocks	111
A1.0 SERDES Link Test Modes	111
A1.1 JRX SERDES Link Test Modes	111
A1.2 JTX SERDES Link Test Modes.....	115
A2.0 Multi-chip Sync & Subclass 1	119
A3.0 Interrupt Service Requests	128
A3.1 DAC Datapath IRQs	128
A3.2 ADC Datapath IRQs	130
A3.3 SERDES Configuration IRQs.....	130
A4.0 Loopback Test Modes	133
A5.0 ADC Signal Monitoring	134
A6.0 SPI Controls.....	140
Hardware Initialization.....	143
SPI Access	143
RESETB Pin Access.....	143
Delay Function	143
Log Function.....	143
API Integration and Build.....	144
Integrating the ADxxxx API into an application	144
1. Implement the HAL Functions	144
2. Include the ADxxxx API Interface header files	144
3. Instantiate ADxxxx Device Handle	145
4. Instantiate user data handle.....	146
5. Instantiate clk_info handle.....	146
6. Create the application.....	146
Application Initialization Sequence.....	146
Building the ADxxxx API Library and Application on ADS9/x platform	147
Use Cases	148
Using External Clock Source for Direct Sample Clock	149
Adding Custom Use case	150

Evaluation Warning: The document was created with Spire.Doc for Python.

Revision History

12 / 03 / 2020 – Rev: 0.0.1
12 / 18 / 2020 – Rev: 0.0.2
12 / 21 / 2020 – Rev: 1.0.7
03 / 03 / 2021 – Rev: 1.1.0

Initial Document.

Formatted API description with functional groups.

Initial Release accompanying API version 1.0.7.

Document Release accompanying API version 1.1.0.

Added Information regarding Use Cases.

Added APIs:

- `adi_adxxxx_adc_ddc_coarse_chip_xfer_set (...)`.
- `adi_adxxxx_adc_ddc_coarse_chip_xfer_status_get (...)`.
- `adi_adxxxx_adc_ddc_fine_chip_xfer_set (...)`.
- `adi_adxxxx_adc_ddc_fine_chip_xfer_status_get (...)`.
- `adi_adxxxx_device_nco_sync_pre (...)`.
- `adi_adxxxx_device_nco_sync_post (...)`.
- `adi_adxxxx_jesd_rx_power_down_des (...)`.
- `adi_adxxxx_jesd_tx_power_down_ser (...)`.
- `adi_adxxxx_device_get_temperature (...)`.

03 / 11 / 2021 – Rev: 1.2.0

Document Release accompanying API version 1.2.0.

Updated APIs:

- `adi_adxxxx_modulation_mux_mode_set`
- `adi_adxxxx_dac_complex_modulation_enable_set`
- `adi_adxxxx_dac_fsc_set`
- `adi_adxxxx_jesd_oneshot_sync`
- `adi_adxxxx_jesd_sysref_input_mode_set`

Added APIs:

- `adi_adxxxx_adc_clk_out_voltage_swing_set`
- `adi_adxxxx_jesd_sysref_oneshot_sync_done_get`
- `adi_adxxxx_jesd_sysref_setup_hold_get`
- `adi_adxxxx_jesd_sysref_fine_superfine_delay_set`
- `adi_adxxxx_jesd_sysref_count_set`
- `adi_adxxxx_jesd_sysref_average_set`
- `adi_adxxxx_jesd_sysref_monitor_phase_get`
- `adi_adxxxx_jesd_sysref_monitor_lmfc_align_error_get`
- `adi_adxxxx_jesd_sysref_monitor_lmfc_align_threshold_set`

Evaluation Warning: The document was created with SpireDoc for Python.

02 / 01 / 2022 – Rev: 1.2.1

Document Release accompanying API version 1.2.1.

Updated APIs:

- `adi_adxxxx_jesd_tx_res_sel_set`

02 / 01 / 2022 – Rev: 1.2.2

Document Release accompanying API version 1.2.2.

06 / 05 / 2022 – Rev: 1.3.0

Document Release accompanying API version 1.3.0.

Updated APIs:

- `adi_adxxxx_jesd_tx_pll_startup`
- `adi_adxxxx_jesd_tx_conv_sel_set`
- `adi_adxxxx_jesd_rx_link_config_set`
- `adi_adxxxx_jesd_tx_link_config_set`
- `adi_adxxxx_jesd_oneshot_sync`
- `adi_adxxxx_jesd_sysref_monitor_phase_get`
- `adi_adxxxx_jesd_sysref_irq_jitter_mux_set`
- `adi_adxxxx_jesd_tx_fbw_sel_set`

Added APIs:

- `adi_adxxxx_adc_data_inversion_dc_coupling_set`
- `adi_adxxxx_adc_offset_timing_calibration_set`
- `adi_adxxxx_adc_offset_calibration_set`
- `adi_adxxxx_adc_gain_calibration_set`
- `adi_adxxxx_jesd_tx_calc_br_ratio`
- `adi_adxxxx_sync_sysref_ctrl`
- `adi_adxxxx_sync_calc_jrx_lmfc_lmfc`
- `adi_adxxxx_sync_calc_jtx_lmfc_lmfc`
- `adi_adxxxx_sync_sysref_frequency_set`

- adi_adxxxx_jesd_tx_lmfc_delay_set
- adi_adxxxx_sync_jrx_tpl_phase_diff_get
- adi_adxxxx_adc_ddc_coarse_sync_next_set
- adi_adxxxx_adc_ddc_coarse_trig_nco_reset_enable_set
- adi_adxxxx_jesd_sysref_irq_enable_set
- adi_adxxxx_jesd_sysref_irq_jitter_mux_set

07 / 15 / 2022 – Rev: 1.3.1
11 / 22 / 2022 – Rev: 1.4.0

Document Release accompanying API version 1.3.1.
Document Release accompanying API version 1.4.0.

Added APIs:

- adi_adxxxx_jesd_cal_bg_cal_pause
- adi_adxxxx_jesd_cal_bg_cal_start
- adi_adxxxx_jesd_rx_qr_two_dim_eye_scan
- adi_adxxxx_jesd_rx_qr_vertical_eye_scan
- adi_adxxxx_jesd_tx_fbw_config_set
- adi_adxxxx_sync_sysref_input_config_set

6 / 1 / 2023 – Rev: 1.5.0

Document Release accompanying API version 1.5.0.

Updated APIs:

- adi_adxxxx_adc_cddc_fddc_jtx_clk_en_via_rxen1_gpio_set to adi_adxxxx_adc_rxengp0_sel_set
- adi_adxxxx_adc_cddc_fddc_jtx_clk_en_via_rxen3_gpio_set to adi_adxxxx_adc_rxengp1_sel_set
- adi_adxxxx_adc_rxen1_gpio_ctrl_set to adi_adxxxx_adc_rxengp0_ctrl_set
- adi_adxxxx_adc_rxen3_gpio_ctrl_set to adi_adxxxx_adc_rxengp1_ctrl_set
- adi_adxxxx_jesd_rx_qr_two_dim_eye_scan
- adi_adxxxx_jesd_rx_phy_prbs_test

Evaluation Warning: The document was created with Spire.Doc for Python.

Added APIs:

- adi_adxxxx_adc_adc0_rxen_pwdn_ctrl_set
- adi_adxxxx_adc_adc1_rxen_pwdn_ctrl_set
- adi_adxxxx_adc_adc2_rxen_pwdn_ctrl_set
- adi_adxxxx_adc_adc3_rxen_pwdn_ctrl_set
- adi_adxxxx_dac_duc_main_nco_ftw_get
- adi_adxxxx_dac_duc_channel_nco_ftw_get
- adi_adxxxx_jesd_rx_ctle_manual_config_get
- adi_adxxxx_jesd_rx_ctle_manual_config_set
- adi_adxxxx_jesd_rx_hr_vertical_eye_scan
- adi_adxxxx_jesd_rx_hr_two_dim_eye_scan

11 / 1 / 2023 – Rev: 1.6.0

Document Release accompanying API version 1.6.0.

Updated APIs:

- adi_adxxxx_adc_analog_input_buffer_set
- adi_adxxxx_adc_path_or_bypass_config
- adi_adxxxx_dac_dll_startup
- adi_adxxxx_dac_duc_main_dsa_set
- adi_adxxxx_device_clk_pll_div_set
- adi_adxxxx_device_startup_tx_or_nco_test
- adi_adxxxx_jesd_rx_pll_startup
- adi_adxxxx_jesd_rx_power_down_des
- adi_adxxxx_jesd_tx_power_down_ser
- adi_adxxxx_jesd_tx_startup_ser
- adi_adxxxx_jesd_tx_link_config_set
- adi_adxxxx_jesd_tx_fbw_config_set

- `adi_adxxxx_jesd_tx_fbw_sel_set`
- `adi_adxxxx_jesd_loopback_mode_set`

Added APIs:

- `adi_adxxxx_dac_duc_sel_sideband_enable_set`
- `adi_adxxxx_dac_dll_power_down`
- `adi_adxxxx_dac_duc_main_dig_gain_set`

10 / 16 / 2024 – Rev 1.7.0

Document Release accompanying API version 1.7.0.

Updated APIs:

- `adi_adxxxx_sync_sysref_frequency_set`

Added APIs:

- `adi_adxxxx_device_startup_nco_test_mode`
- `adi_adxxxx_adc_pfir_coeff_table_load_set`

Evaluation Warning: The document was created with Spire.Doc for Python.

Introduction

Purpose

This document serves as a programmer's reference for using and utilizing various aspects of the Application Program Interface (API) library targeting the AD908x and AD998x (MxFE™) family of software defined, direct RF sampling transceivers. Additionally, this document encompasses the definitions of the stand-alone DAC only (AD9177) and ADC only (AD9207/AD9209) variants as the API packages for these variants are a subset of all the API functions included in the larger MxFE™ family products.

It describes the general structure of the AD908x / AD998x API superset library, provides a detailed list of the API functions and its associated data structures, macros, and definitions. To generalize the API functions, the prefix ADxxxx is used in which xxxx will be 9081, 9082, 9986 or 9988 for the MxFE™ products or 9177 / (9207/9209) for the DAC only / ADC only variants.

Scope

This document targets API libraries for the devices listed in Table 1. A source code package is available for each of the devices.

Target Name	Device	Device Description
AD9081		4x 16-Bit, 12 GSPS RF DAC cores and 4x 12-Bit, 4 GSPS RF ADC cores.
AD9082		4x 16-Bit, 12 GSPS RF DAC cores and 2x 12-Bit, 6 GSPS RF ADC cores.
AD9986		4x 16-Bit, 12 GSPS RF DAC cores and 2x 12-Bit, 6 GSPS RF ADC cores.
AD9988		4x 16-Bit, 12 GSPS RF DAC cores and 4x 12-Bit, 4 GSPS RF ADC cores.
AD9207 ¹		2x 12-Bit, 6 GSPS ADC cores.
AD9209 ²		4x 12-Bit, 4 GSPS RF ADC cores.
AD9177 ³		4x 16-Bit, 12 GSPS RF DAC cores.

Table 1. List of Target Devices supported by this API.

Evaluation Warning: The document was created with Spire Doc for Python. Refer to General API Specification, Integration and Porting document for an overview on the API architecture and some common source files which are shared by most of the ADI device for which an API will be provided. The General API Specification, Integration and Porting Guide must be consulted in conjunction with this user guide to achieve successful understanding in using API functions and help in integrating the API with user platform.

Disclaimer

This is a preliminary pre-release version of API and documentation. All information is subject to change.

The software and any related information and/or advice is provided on and "AS IS" basis, without representations, guarantees or warranties of any kind, express or implied, oral or written, including without limitation warranties of merchantability fitness for a particular purpose, title and non-infringement. Please refer to the Software License Agreement applied to the source code for full details.

¹ For the AD9207 ADC only device, any DAC related API methods and definitions are not included in the AD9207 specific API source code package. Only a subset of API methods are included in the source code that are relevant to this product variant.

² For the AD9209 ADC only device, any DAC related API methods and definitions are not included in the AD9209 specific API source code package. Only a subset of API methods are included in the source code that are relevant to this product variant.

³ For the AD9177 DAC only device, any ADC related API methods and definitions are not included in the AD9177 specific API source code package.

Only a subset of API methods are included in the source code that are relevant to this product variant.

adi_adxxxx.h

The ADxxxx API library has a main interface header file **adi_adxxxx.h** that defines the software interface to the ADxxxx device. It is in **src/ adxxxx_api/ adi_inc/** folder. It consists of a list of API functions and several data structures and enumerations to describe the configurations and settings that are configurable on the target device.

Data Structures

The header file also defines the device handle [adi_adxxxx_device_t](#) which is a data structure that acts a software reference to an instance of an ADxxxx device. This handle maintains a reference to HAL functions ([adi_adxxxx_hal_t](#)), configuration status of the chip ([adi_adxxxx_info_t](#)) and a SERDES structure ([adi_adxxxx_serdes_settings_t](#)) that stores the settings for configuring the JESD Rx / Tx lanes required to operate the chip. Table 2 summarizes these handles.

Structure Member	Description	User Read / Write Access	Required by API
hal_info	A structure maintaining a reference to the application's implementation of the HAL functions required by the API. See Table 3 for information in individual components of adi_adxxxx_hal_t .	Read / Write	Required
dev_info	A structure maintaining a reference to the configuration status of the device.	None	Required
serdes_info	A structure that holds the configuration setting for JESD.	Read / Write	Required
clk_info	A structure that holds the configuration setting for SYSREF signal source control.	Read / Write	Optional

Table 2. ADxxxx Device Handle Members.

hal_info

The HAL specific reference shall be initialized by the client application. Initialize the application with client specific data. The subsequent sections provide the information for correct integration.

All the platform specific members of the HAL structure must be configured by the client application prior to calling any API with the handle. A summary of the individual component members of HAL handle, **adi_adxxxx_hal_t**, supported by the API are listed in Table 3.

Structure Member	Description	User Read / Write Access	Required by API
user_data	Void Pointer to a user defined data structure. Shall be passed to all HAL functions.	Read / Write	Required
hw_open	Function pointer to platform initialization function for ADxxxx hardware.	Read / Write	Optional
hw_close	Function pointer to platform shutdown function for ADxxxx hardware.	Read / Write	Optional
sdo	Configure SPI interface in 3 / 4 wire mode.	Read / Write	Required
msb	Configure SPI frame to be MSB / LSB first.	Read / Write	Required
addr_inc	Configure SPI interface address increment.	Read / Write	Required
spi_xfer	Function pointer to platform specific SPI data transfer function for the target device.	Read / Write	Required
reset_en_pin_ctrl	Function Pointer to ~RESET pin control function.	Read / Write	Optional
log_write	Function pointer to log error, warnings and debug messages.	Read / Write	Optional
delay_us	Function pointer to delay function.	Read / Write	Required

Table 3. Members of **adi_adL5580_hal_t**. ADxxxx device HAL handle.

dev_info

This data structure stores device information like its revision number along with clock setting for device reference, DAC and ADC.

serdes_info

The structure members in this data structure store the settings required to configure JESD204B / C lanes.

Also, for full description and details on configuration of **HAL Function Pointer Data Types**, refer the General API Specification document.

clk_info

This data structure stores SYSREF clock source information. It is optional unless working in Subclass 1.

Structure Member	Description	User Read/Write Access	Required by API
sysref_clk	Void pointer to define clock source for SYSREF signal	Read / Write	Optional
sysref_ctrl	Function pointer to SYSREF pulse request function	Read / Write	Optional
sysref_mode	Configure SYSREF to continuous or single pulse mode	Read / Write	Required

API Overview Block Diagram

To set up the ADxxxx MxFE™ products, there are a variety of system high level API function calls that facilitate the setup of the part in a variety of conditions. These function calls cover the configuration of all the key feature blocks of the product and abstract the details of the required specific sequences to set up the chip properly to minimize the burden on the user to do all steps manually.

Figure 1 shows the general overview of the whole system bring-up surrounding the MxFE™ products. The blocks in green represent the stages of the hardware and system configuration outside of the ADxxxx that set up the environment around the product. These steps should be adapted based on the user's setup and reference some of the example code and steps needed when using the Analog Devices evaluation platform (ADS9v2 FPGA board with the MxFE™ evaluation board). The blocks in blue reference the top system level API functions that are called as part of the example standalone application to set up the ADxxxx products. The example standalone application, which is included in the source code package under **src/adxxxx_app** folder, provides a set of pre-configured use cases that set up various conditions by calling these system top level API. For most use cases, the startup process listed out below is enough to get the MxFE™ configured properly for integration in any system.

This startup process is broken up into 4 basic functional groups of configurations: initialization and clocking, transmit datapath setup, receive datapath setup, and SERDES link establishment. The API function calls needed for each section are shown underneath each of these steps vertically in **Figure 1** and described in detail in their respective x.0 sections in the [ADxxxx API Functions Detailed Descriptions](#) section of this document.

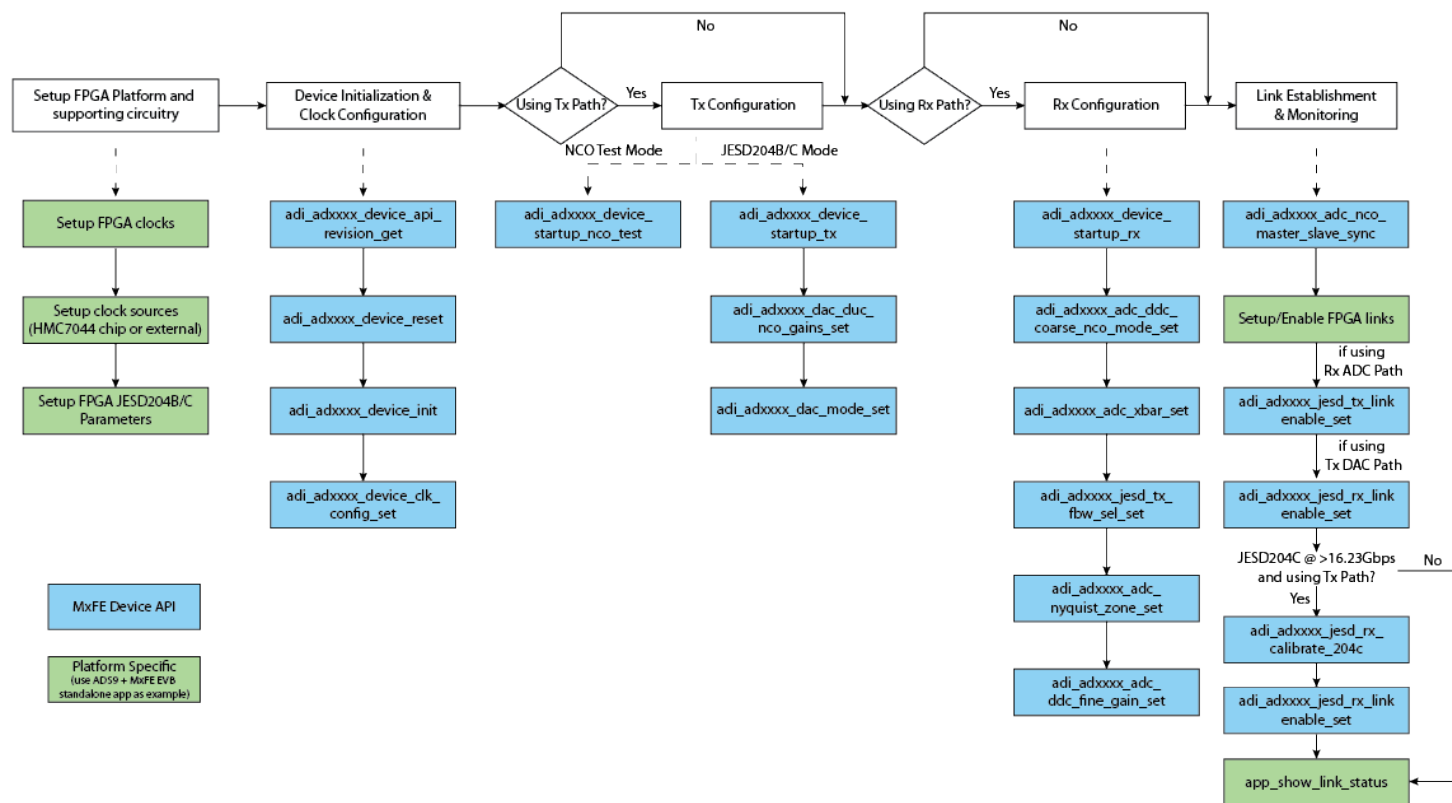


Figure 1. System High Level API Calls Overview

While the setup flow shown in Figure 1 sets up the majority of the basic functional blocks in the MxFE™ products, there are block-level API function calls underneath each high-level section that can additionally be called independently from some of the higher level API calls to tweak settings in various functional blocks of the chip. These are described in more detail in the sub-sections for each of the 4 basic functional groups and the input parameters shown available for the user's configuration. All source code for all the API methods are provided as part of the API source code package and show the details of how each block configuration is executed.

Evaluation Warning: The document was created with Spire.Doc for Python.